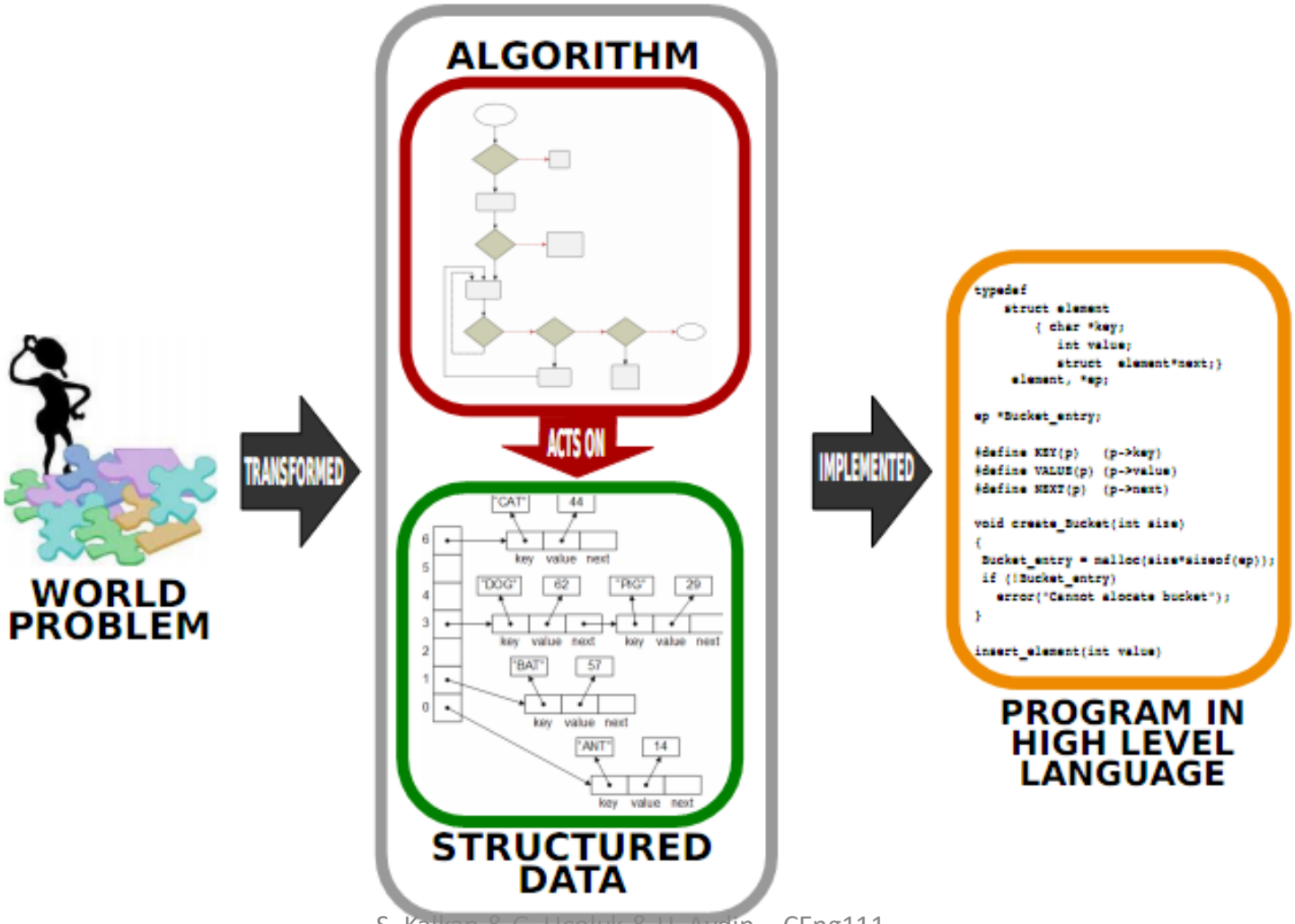


CEng 111 – Fall 2026
Week 6a

Credit: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.

Previously on CENG111!

Design of a solution





Previously on CENG111!

Binary Representation of Numeric Information (continued)

- Sign/magnitude notation

  1 101 = - 5

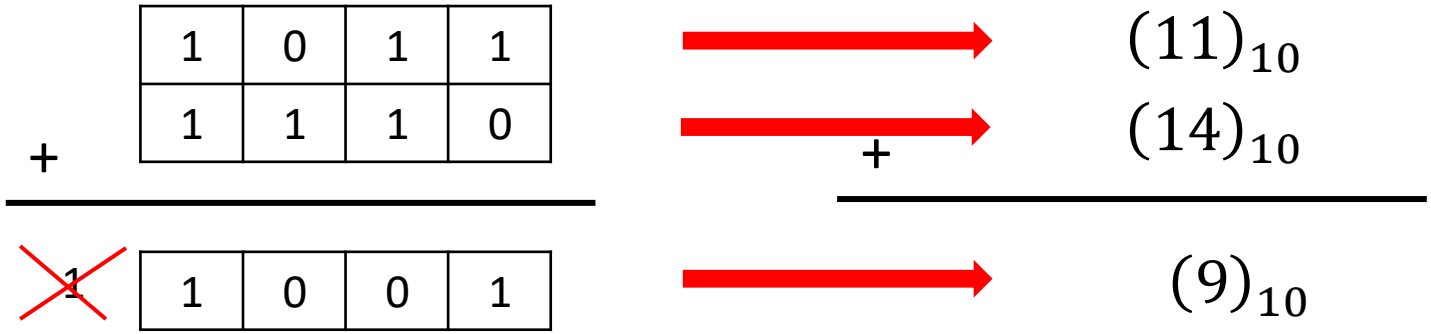
  0 101 = + 5

- Problems:
  - Two different representations for 0:
    - 1 000 = - 0
    - 0 000 = +0
  - Addition & subtraction require a watch for the sign! Otherwise, you get incorrect results:
    - 0 010 (+2) + 1 010 (-2) = 1 100 (-4)

Arithmetic in Computers is Modular

Previously on CENG111!

Let's add two numbers in binary
(Assume that there is no sign bit)



In other words:

- Numbers larger than or equal to 16 (2^4) are discarded in a 4-bit representation.
- Therefore, $11 + 14$ yields 9 in this 4-bit representation.
- This is actually modular arithmetic:

$$11 + 14 \; mod \; 16 \equiv 9 \; mod \; 16$$

For more info about modular arithmetic: https://brilliant.org/wiki/modular-arithmetic/



Binary Representation of Numeric Information (continued)

Previously on CENG111

METU Computer Engineering

- Shortcut to convert from "two's complement" :
  - If the leading bit is zero, no need to convert.
  - If the leading bit is one, invert the number and add 1.
- What is our range?
  - With 2's complement we can represent numbers from -2^{N-1} to $2^{N-1} - 1$ using N bits.
  - 8 bits: -128 to +127.
  - 16 bits: -32,768 to +32,767.
  - 32 bits: -2,147,483,648 to +2,147,483,647.

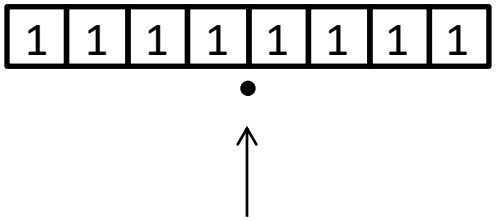
Binary Number	Decimal Value	Value in Two's Complement
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1



Previously on CENG111!

Binary Representation of Real Numbers

- Approach 1: Use fixed-point
  - Similar to integers, except that there is a decimal point.
  - E.g.: using 8 bits:

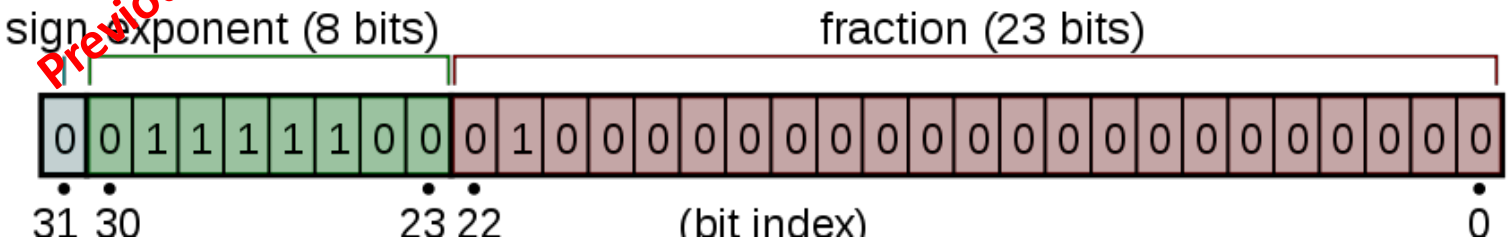


$= 1\times2^3 + 1\times2^2 + 1\times2^1 + 1\times2^0 + 1\times2^{-1} + 1\times2^{-2} + 1\times2^{-3} + 1\times2^{-4}$

$= 15.9375$

IEEE 32bit Floating-Point Number Representation

Previously on CENG111!



$$= (-1)^{\text{sign}}(1.b_{-1}b_{-2}...b_{-23})_2 \times 2^{e-127}$$

- M x 2^E $(2 - 2^{-23}) \times 2^{127}$
- Exponent (E): 8 bits
  - Add 127 to the exponent value before storing it
  - **E can be 0 to 255 with 127 representing the real number zero (0).**
- Fraction (M - Mantissa): 23 bits
- $2^{128} = 3.4028237 \times 10^{38}$

METU Computer Engineering

IEEE 32bit Floating-Point Number Representation

Previously on CENG111!

- Now consider 4.1:
  - 4 => $(100)_2$
  - 0.1 =>
    - x 2 = 0.2 = 0 + 0.2
    - x 2 = 0.4 = 0 + 0.4
    - x 2 = 0.8 = 0 + 0.8
    - x 2 = 1.6 = 1 + 0.6
    - x 2 = 1.2 = 1 + 0.2
    - x 2 = 0.4 = 0 + 0.4
    - x 2 = 0.8 = 0 + 0.8
    - .....

- So,
  - Representing a fraction which is a multiple of $1/2^n$ is lossless.
  - Representing a fraction which is not a multiple of $1/2^n$ leads to accuracy loss.



Today

- Binary representation
  - Characters
  - Other types of information



Administrative Notes

- Midterm Exam
  - Date: 20 November, 18:00
- Lab Exams
  - Lab Exam 1: 8 November, Sunday, 13:00
  - Lab Exam 2: 6 December, Sunday, 13:00
  - Lab Exam 3: 20 December, Sunday, 13:00
  - Lab Exam 4: 3 January, Sunday, 13:00



BINARY REPRESENTATION OF TEXT ETC.



Binary Representation of Textual Information

METU Computer Engineering

- Characters are mapped onto binary numbers
  - ASCII (American Standard Code for Information Interchange) code set
    - Originally: 7 bits per character; 128 character codes
  - Unicode code set
    - 16 bits per character
  - UTF-8 (Universal Character Set Transformation Format) code set.
    - Variable number of 8-bits.



ASCII
7 bits long

Dec	Bin	Char	Dec	Bin	Char	Dec	Bin	Char	Dec	Bin	Char
0	0000 0000	[NUL]	32	0010 0000	space	64	0100 0000	@	96	0110 0000	`
1	0000 0001	[SOH]	33	0010 0001	!	65	0100 0001	A	97	0110 0001	a
2	0000 0010	[STX]	34	0010 0010	"	66	0100 0010	B	98	0110 0010	b
3	0000 0011	[ETX]	35	0010 0011	#	67	0100 0011	C	99	0110 0011	c
4	0000 0100	[EOT]	36	0010 0100	$	68	0100 0100	D	100	0110 0100	d
5	0000 0101	[ENQ]	37	0010 0101	%	69	0100 0101	E	101	0110 0101	e
6	0000 0110	[ACK]	38	0010 0110	&	70	0100 0110	F	102	0110 0110	f
7	0000 0111	[BEL]	39	0010 0111	'	71	0100 0111	G	103	0110 0111	g
8	0000 1000	[BS]	40	0010 1000	(	72	0100 1000	H	104	0110 1000	h
9	0000 1001	[TAB]	41	0010 1001	)	73	0100 1001	I	105	0110 1001	i
10	0000 1010	[LF]	42	0010 1010	*	74	0100 1010	J	106	0110 1010	j
11	0000 1011	[VT]	43	0010 1011	+	75	0100 1011	K	107	0110 1011	k
12	0000 1100	[FF]	44	0010 1100	,	76	0100 1100	L	108	0110 1100	l
13	0000 1101	[CR]	45	0010 1101	-	77	0100 1101	M	109	0110 1101	m
14	0000 1110	[SO]	46	0010 1110	.	78	0100 1110	N	110	0110 1110	n
15	0000 1111	[SI]	47	0010 1111	/	79	0100 1111	O	111	0110 1111	o
16	0001 0000	[DLE]	48	0011 0000	0	80	0101 0000	P	112	0111 0000	p
17	0001 0001	[DC1]	49	0011 0001	1	81	0101 0001	Q	113	0111 0001	q
18	0001 0010	[DC2]	50	0011 0010	2	82	0101 0010	R	114	0111 0010	r
19	0001 0011	[DC3]	51	0011 0011	3	83	0101 0011	S	115	0111 0011	s
20	0001 0100	[DC4]	52	0011 0100	4	84	0101 0100	T	116	0111 0100	t
21	0001 0101	[NAK]	53	0011 0101	5	85	0101 0101	U	117	0111 0101	u
22	0001 0110	[SYN]	54	0011 0110	6	86	0101 0110	V	118	0111 0110	v
23	0001 0111	[ETB]	55	0011 0111	7	87	0101 0111	W	119	0111 0111	w
24	0001 1000	[CAN]	56	0011 1000	8	88	0101 1000	X	120	0111 1000	x
25	0001 1001	[EM]	57	0011 1001	9	89	0101 1001	Y	121	0111 1001	y
26	0001 1010	[SUB]	58	0011 1010	:	90	0101 1010	Z	122	0111 1010	z
27	0001 1011	[ESC]	59	0011 1011	;	91	0101 1011	[	123	0111 1011	{
28	0001 1100	[FS]	60	0011 1100	<	92	0101 1100	\	124	0111 1100	|
29	0001 1101	[GS]	61	0011 1101	=	93	0101 1101	]	125	0111 1101	}
30	0001 1110	[RS]	62	0011 1110	>	94	0101 1110	^	126	0111 1110	~
31	0001 1111	[US]	63	0011 1111	?	95	0101 1111	_	127	0111 1111	[DEL]

Some books make a mistake about the number of characters in the ASCII table.

To store the 4-character string "BAD!" in memory, the computer would store the binary representation of each individual character using the above 8-bit code.

$$\text{BAD!} = \underbrace{00000010}_{\text{B}}\ \underbrace{00000001}_{\text{A}}\ \underbrace{00000100}_{\text{D}}\ \underbrace{10000001}_{\text{!}}$$

We have indicated above that the 8-bit numeric quantity 10000001 is interpreted as the character "!". However, as we mentioned earlier, the only way a computer knows that the 8-bit value 10000001 represents the symbol "!" and not the unsigned integer value 129 (128 + 1) or the signed integer value –1 (sign bit = negative, magnitude is 1) is by the context in which it is used. If these 8 bits are sent to a display device that expects to be given characters, then this value will be interpreted as an "!". If, on the other hand, this 8-bit value is sent to an arithmetic unit that adds unsigned numbers, then it will be interpreted as a 129 in order to make the addition operation meaningful.

To facilitate the exchange of textual information, such as word processing documents and electronic mail, between computer systems, it would be most helpful if everyone used the same code mapping. Fortunately, this is pretty much the case. Currently the most widely used code for representing characters internally in a computer system is called **ASCII**, an acronym for the *A*merican *S*tandard *C*ode for *I*nformation *I*nterchange. ASCII is an international standard for representing textual information in the majority of computers. It uses 8 bits to represent each character, so it is able to encode a total of $2^8 = 256$ different characters. These are assigned the integer values 0 to 255. However, only the numbers 32 to 126 have been assigned so far to printable characters. The remainder either are unassigned or are used for nonprinting control characters such as tab, form feed, and return. Figure 4.3 shows the ASCII conversion table for the numerical values 32–126.

However, a new code set called **UNICODE** is rapidly gaining popularity because it uses a 16-bit representation for characters rather than the 8-bit format of ASCII. This means that it is able to represent $2^{16} = 65{,}536$ unique characters instead of the $2^8 = 256$ of ASCII. It may seem like 256 characters are more than enough to represent all the textual symbols that we would ever need—for example, 26 uppercase letters, 26 lowercase letters, 10 digits, and a few dozen special symbols, such as +=–{}][\;'"?><.,;%$#@. Add that all together and it still totals only about 100 symbols, far less than the 256 that can be represented in ASCII. However, that is true only if we limit our work to Arabic numerals and the Roman alphabet. The world grows more connected all the time—helped along by computers, networks, and the Web—and it is critically important that computers represent and exchange textual information using alphabets in addition to these 26 letters and 10 digits. When we start assigning codes to symbols drawn from alphabets such as Russian, Arabic, Chinese, Hebrew, Greek, Thai, Bengali, and Braille, as well as mathematical symbols and special linguistic marks such as tilde, umlaut, and accent grave, it becomes clear that ASCII does not have nearly enough room to represent them all. However, UNICODE, with space for over 65,000 symbols, is large enough to accommodate all these symbols and many more to come. In fact, UNICODE has defined standard code mappings for more

UTF-8 Illustrated

Bits	Last code point	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
7	U+007F	0xxxxxxx					
11	U+07FF	110xxxxx	10xxxxxx				
16	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx			
21	U+1FFFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx		
26	U+3FFFFFF	111110xx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	
31	U+7FFFFFFF	1111110x	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx

Character		Binary code	Binary UTF-8
$	U+0024	0100100	00100100
¢	U+00A2	00010100010	11000010 10100010
€	U+20AC	0010000010101100	11100010 10000010 10101100
𤭢	U+24B62	000100100101101100010	11110000 10100100 10101101 10100010



How about a text?

- Text in a computer has two alternative representations:

1. A fixed-length number representing the length of the text followed by the binary values of the characters in the text.
   - Ex: “ABC” =>

   00000011 01000001 01000001 01000001 (3 ‘A’ ‘B’ ‘C’)

2. Binary values of the characters in the text ended with a unique marker, like “00000000” which has no value in the ASCII table.
   - Ex: “ABC” =>

   01000001 01000001 01000001 00000000 (‘A’ ‘B’ ‘C’ END)



Binary Representation of Sound and Images

- Multimedia data is sampled to store a digital form, with or without detectable differences
- Representing sound data
  - Sound data must be digitized for storage in a computer
  - Digitizing means periodic sampling of amplitude values



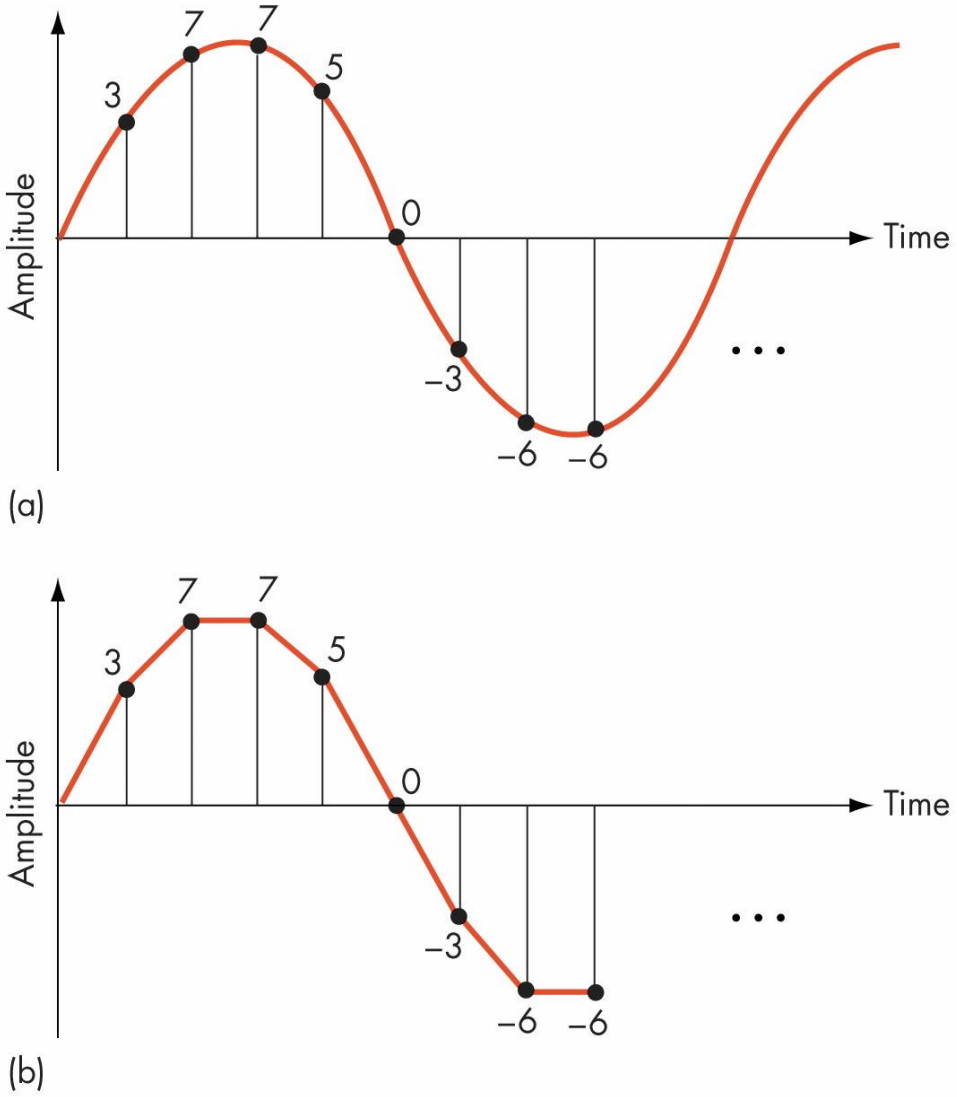
Binary Representation of Sound and Images (continued)

- From samples, original sound may be approximated
- To improve the approximation:
  - Sample more frequently (*increase sampling rate)*
  - Use more bits for each sample value *(↑ bit depth)*

Digitization of an Analog Signal

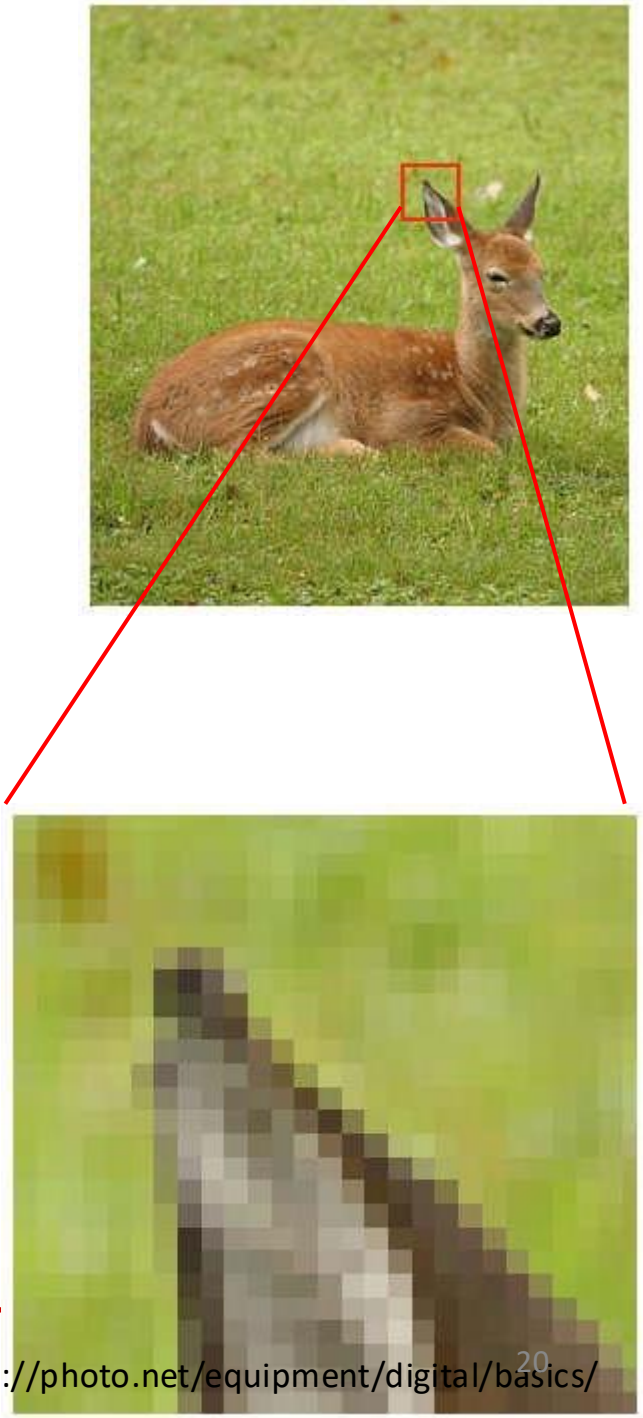
(a) Sampling the Original Signal

(b) Recreating the Signal from the Sampled Values



Binary Representation of Sound and Images (continued)

- Representing image data
  - Images are sampled by reading color and intensity values at even intervals across the image
  - Each sampled point is a pixel
  - Image quality depends on number of bits at each pixel
  - More image information: http://cat.xula.edu/tutorials/imaging/grayscale.php



http://photo.net/equipment/digital/basics/

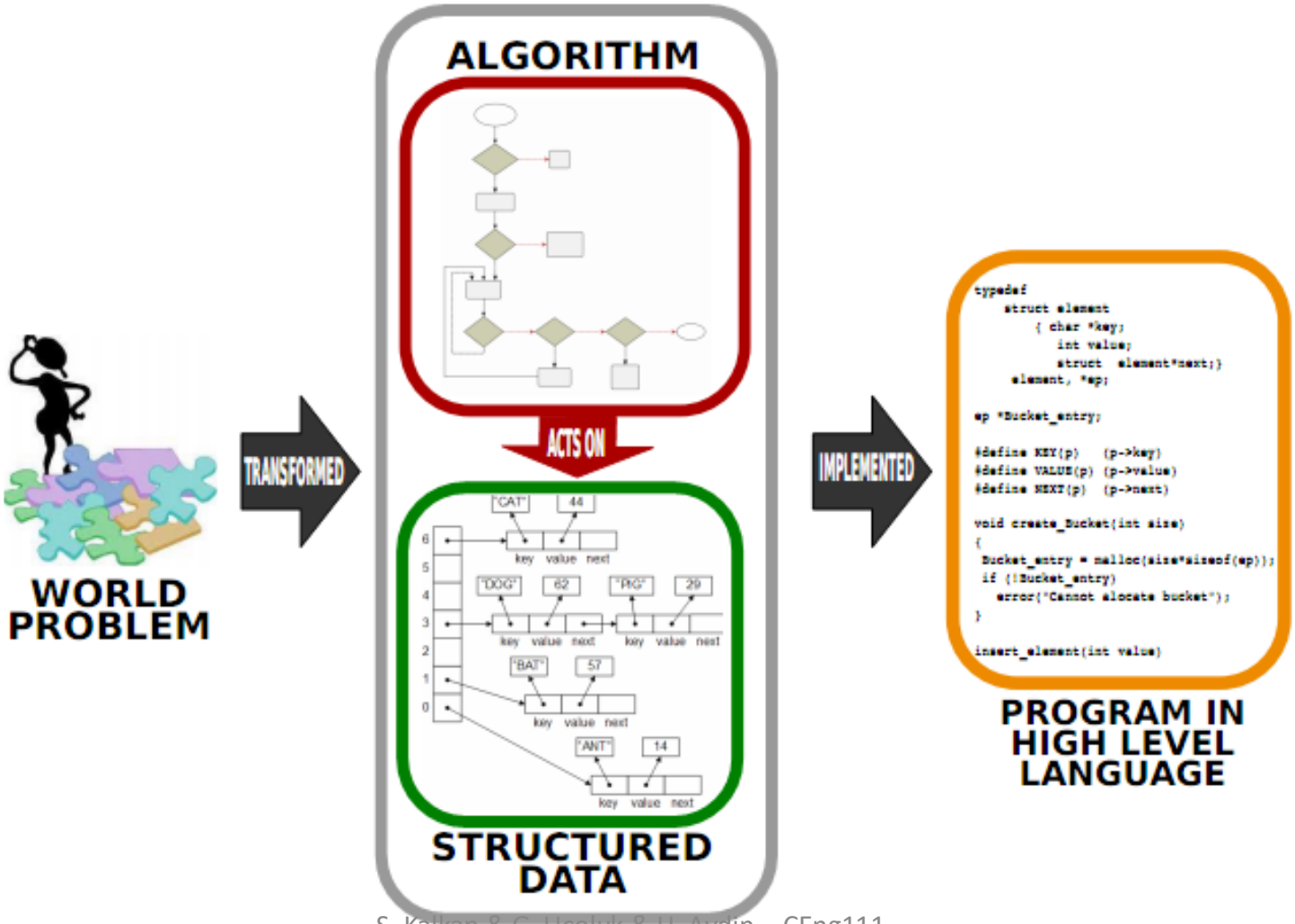


The Reliability of Binary Representation

- Electronic devices are most reliable in a bistable environment
- Bistable environment
  - Distinguishing only two electronic states
    - Current flowing or not, or
    - Direction of flow
- Computers are bistable: hence binary representations
- See the following video for contrasting the design of a binary computer and a ternary one:

  https://www.youtube.com/watch?v=fXwSFhUVFmE

Design of a solution



What is data?

- **Data**: Information to be processed to solve a problem.
- Identify the data for the following example problems:
  - Find all wheat growing areas in a terrestrial satellite image.
  - Given the homework, lab and examination grades of a class, calculate the letter grades.
  - Alter the amplitude of a sound recording for various frequencies.
  - Extrapolate China's population for the year 2040 based on the change in the population growth rate up to this time.
  - Compute the launch date and the trajectory for a space probe so that it will pass by the outermost planets in the closest proximity.
  - Compute the layout of the internals of a CPU so that the total wiring distance is minimized.
  - Find the cheapest flight plan from A to B, for given intervals for arrival and departure dates.
  - Simulate a war between two land forces, given (i) the attack and the defense plans, (ii) the inventories and (iii) other attributes of both forces.



What is data?

- CPU can only understand two types of data:
  - Integers,
  - Floating points.
- The following are not directly understandable by a CPU:
  - Characters (‘a’, ‘A’, ‘2’, …)
  - Strings (“apple”, “banana”, …)
  - Complex Numbers
  - Matrices
  - Vectors
- But, programming languages can implement these data types.



Basic Data Types

- Integers
  - Full support from the CPU
  - Fast processing
- Floating Points
  - Support from the CPU with some precision loss
  - Slower compared to integer processing due to “interpretation”



Problems due to precision loss

- What has precision loss is misleading: 0.9 has precision loss whereas 0.9375 does not!
  - 0.9375 = 0.5 + 0.25 + 0.125 + 0.0625
  - Therefore: $(0.9375)_{10} = (0.1111)_2$
- Imprecise representations are round-offs!
- If you have a lot of round-offs, you might be in trouble!



Problems due to precision loss (Example 1)

- 1.0023 - 1.0567
  - Result: -0.05440000000000004
- 1000.0023 - 1000.0567
  - Result: -0.05439999999986903
- Why?
  - Since the floating point representation is based on shifting the bits in the mantissa, the two calculations are not equivalent in a computer



Problems due to precision loss (Example 2)

- π = 3.1415926535897931….
  - sin(π) should be zero
  - But it is not: $1.2246467991473532 \times 10^{-16}$

Problems due to precision loss (Example 3)

- Associativity in mathematics:
  - $(a+b) + c = a + (b + c)$
  - This does not hold in floating-point arithmetic:

set $a = 1234.567$, $b = 45.67834$ and $c = 0.0004$:
$(a + b) + c$ will produce 1280.2457399999998,
$a + (b + c)$ will produce 1280.2457400000001.

METU Computer Engineering



What to do with precision loss?

- If you can transform the problem to the integer domain, do so.
  - Refrain from using floating points as much as you can.
- Use the most precise type of floating point of the high-level language you are using.
  - Use only less precision floating points if you are short in memory.



What to do with precision loss?

METU Computer Engineering

- If you have two numbers that are magnitude-wise incomparable, you are likely to lose the contribution of the smaller one. That will yield unexpected results when you repeat the addition in a loop where the looping is so much that the accumulation of the smaller is expected to become significant. It will not.
- The contrary can also happen. Slight inaccuracies accumulate in loops to significant magnitudes and yield non-sense values.
- You better use well known, decent floating point libraries instead of coding floating point algorithms by yourself.



Integers in Python

METU Computer Engineering

- Python provides `int` type.

```
>>> type(3)
<type 'int'>
>>> type(3+4)
<type 'int'>
>>>
```

- For big integers, Python v2 has the `long` type:

```
>>> type(3L)
<type 'long'>
>>> type(3L+4L)
<type 'long'>
>>>
```

`int` is limited by the hardware whereas `long` type is unlimited in Python.



Floating Points in Python

- Python provides `float` type.

```
>>> type(3.4)
<type 'float'>
>>>
```

```
>>> 3.4+4.3
7.7
>>> 3.4 / 4.3
0.7906976744186046
```

See the following document for how Python handles precision loss:
https://docs.python.org/3/tutorial/floatingpoint.html



Simple Operations with Numerical Values in Python

METU Computer Engineering

Operator	Operator Type	Description
+	Binary	Addition of two operands
-	Binary	Subtraction of two operands
-	Unary	Negated value of the operand
*	Binary	Multiplication of two operands
/	Binary	Division of two operands
**	Binary	Exponentiation of two operands (Ex: `x**y` = x^y)

- **abs**(x): Absolute value of x.
- **round**(Float): Rounded value of float.
- **int**(Number), **long**(Number), **float**(Number): Conversion between numerical values of different types.



Integer division vs. float division

Python v2

- "/" ("classic division")
  - Removes the fraction if both parameters are integers

```
>>> 3 / 4 # returns  0
```

  - Performs "true division" if either of the parameters if a float

```
>>> 3 / 4.0  # returns 0.75
```

- "//" ("floor division")
  - Removes the fraction all the time

```
>>> 3 // 4 # returns 0
>>> 3 // 4.0 # returns 0.0
```

Python v3

- "/" ("true division")
  - Performs true division all the time

```
>>> 3 / 4 # returns  0.75
>>> 3 / 4.0  # returns 0.75
```

- "//" ("floor division")
  - Removes the fraction all the time

```
>>> 3 // 4 # returns 0
>>> 3 // 4.0 # returns 0.0
```

Characters

- We need characters to represent textual data:
  - Characters: ‘A’, …, ‘Z’, ‘0’, .., ‘9’, ‘a’, …, ‘z’ … etc.
- Computer uses the ASCII table for converting characters to binary numbers:

SYN	00010110	,	00101100	B	01000010	X	01011000	n	01101110
ETB	00010111	-	00101101	C	01000011	Y	01011001	o	01101111
CAN	00011000	.	00101110	D	01000100	Z	01011010	p	01110000
EM	00011001	/	00101111	E	01000101	(	01011011	q	01110001
SUB	00011010	0	00110000	F	01000110	\	01011100	r	01110010
ESC	00011011	1	00110001	G	01000111	)	01011101	s	01110011
FS	00011100	2	00110010	H	01001000	^	01011110	t	01110100
GS	00011101	3	00110011	I	01001001	_	01011111	u	01110101
RS	00011110	4	00110100	J	01001010	‘	01100000	v	01110110
US	00011111	5	00110101	K	01001011	a	01100001	w	01110111
SPC	00100000	6	00110110	L	01001100	b	01100010	x	01111000
!	00100001	7	00110111	M	01001101	c	01100011	y	01111001
"	00100010	8	00111000	N	01001110	d	01100100	z	01111010
#	00100011	9	00111001	O	01001111	e	01100101	}	01111011
$	00100100	:	00111010	P	01010000	f	01100110	|	01111100
%	00100101	;	00111011	Q	01010001	g	01100111	}	01111101
&	00100110	<	00111100	R	01010010	h	01101000	~	01111110
’	00100111	=	00111101	S	01010011	i	01101001	DEL	01111111
(	00101000	>	00111110	T	01010100	j	01101010		
)	00101001	?	00111111	U	01010101	k	01101011		
*	00101010	@	01000000	V	01010110	l	01101100		
+	00101011	A	01000001	W	01010111	m	01101101		

A portion of the ASCII table.



Characters in Python

- Python does not have a separate data type for characters!
- However, one-character strings can be treated like characters:
  - **ord**(One_Char_String) : returns the ASCII value of the character in One_Char_String.
    - Ex: ord("A") returns 65.
  - **chr**(ASCII_value) : returns the character in a string that has the ASCII value ASCII_value:
    - Ex: chr(66) returns "B"



METU Computer Engineering

Boolean Values

- The CPU often needs to compare numbers, or data:
  - $3 >^{?} 4$
  - $125 =^{?} 1000/8$
  - $3 \leq^{?} 12345.34545/12324356.0$
- We have the truth values for representing the answers to such comparisons:
  - If correct: TRUE, True, true, T, 1
  - If not correct: FALSE, False, false, F, 0



Boolean Values in Python

- Python provides the `bool` data type for boolean values.
- `bool` data type can take `True` or `False` as values.

```
>>> 3 > 4
False
>>> type(3 > 4)
<type 'bool'>
```

```
>>> True == 2
False
>>> True == 1
True
>>> False == 0
True
>>> False == 1
False
```

Try this:

```
>>> True == bool(2)
```

not operator:

```
not True
not 4 > 3
```

Structured Data

METU Computer Engineering

- If you have lots and lots of one type of data (for example, the ages of all the people in Turkey):
  - You can store them into memory consecutively (supported by most PLs)
    - This is called *arrays*.
  - Easy to access an element. Nth element:
    - <Starting-address>+ (N-1)*<Word Width>
    - Ex: 2nd element is at 128 + (2-1) * 4 = 132

